Individual Contribution Report

Student Name: Jhetro Jade Tumayao

Group Name: AIRHA

Assigned Role: User / System Developer

1. What did I contribute to the project?

- Designed and built the complete user authentication and role management module in users.py.
- Implemented secure credential storage using standard cryptographic hashing.
- Developed session control methods for user login, logout, active role detection, and user listing.

2. What features did I personally develop?

- **_hash(password)**: Utility function that computes an SHA-256 hex digest for password
- **UserManager.__init__()**: In-memory dictionary storing account records (admin and student), their corresponding roles (Administrator and Standard User), and hashed passwords.
- **authenticate(username, password)**: Validates credentials by comparing input password hashes against saved account hashes and sets self.current upon success.
- **login_prompt()**: Console interface handling interactive credential prompts up to MAX_ATTEMPTS = 3, featuring hidden terminal entry via getpass, fallback input handling, and dynamic attempt counter feedback.
- **logout()**: Resets the active session by setting self.current to None.
- **role()**: Returns the role string (Administrator, Standard User, or "None") for the active user session.
- **list_users()**: Generates a list of tuples containing all registered usernames and their assigned roles.

3. What problems did I encounter?

- **Plaintext Security:** Storing raw password strings directly in the source code presented major security vulnerabilities.

- **Console Visibility:** Using standard input() for passwords shows characters publicly on-screen during typing.
- **Console Compatibility:** In non-standard terminal environments or IDE consoles, getpass.getpass() can raise exceptions.
- **Session Persistence:** Keeping track of logged-in user authority across multiple system commands required clean state isolation.

4. How did I solve those problems?

- **SHA-256 Digesting:** Applied Python's hashlib.sha256() so passwords are only evaluated as SHA-256 hex hashes.
- **Masked Entry & Fallback Handling:** Integrated getpass.getpass() for obscured typing inside a try-except block, defaulting gracefully to input() if an unexpected terminal exception occurs.
- **Encapsulated Class State:** Stored active session details in self.current within the UserManager class to allow methods like role() and logout() to operate dynamically.

5. What did I learn from this project?

- **Authentication Basics:** Gained practical knowledge of implementing secure password hashing and basic credential validation workflows.
- **Defensive CLI Design:** Learned how to build robust console interactions with retry limits, exception handling, and masked inputs.
- **Object-Oriented Design:** Strengthened skills in state management using Python classes to handle system user sessions clean and modularly

6.EVIDENCE OF MY CONTRIBUTION

```python
# Logins
import getpass
import hashlib


def _hash(password):
    return hashlib.sha256(password.encode("utf-8")).hexdigest()


class UserManager:
    MAX_ATTEMPTS = 3

    def __init__(self):
        # Password
        self._accounts = {
            "admin": {"hash": _hash("admin123"), "role": "Administrator"},
            "student": {"hash": _hash("student123"), "role": "Standard User"},
        }
        self.current = None

    def authenticate(self, username, password):
        account = self._accounts.get(username)
        if account and account["hash"] == _hash(password):
            self.current = username
            return True
        return False

    def login_prompt(self):
        """Ask for credentials. Returns True on success, False after too many failures."""
        for attempt in range(1, self.MAX_ATTEMPTS + 1):
            username = input("Username: ").strip()
            try:
                password = getpass.getpass("Password: ")
            except (EOFError, KeyboardInterrupt):
                raise
            except Exception:
```